數位系統設計 LAB 2 第 16 組

成員: 11411133 曾俊睿、11411142 林威廷、11411172 李宜倫

1. 請各組了解 Q-M 化簡法，並且運用 AI 工具設計 Q-M 化簡法後，了解相關程式概念，繳交程式說明與成果。

程式說明：

使用 python 做軟體端 Q-M 化簡後，再用 VHDL 的資料流（Dataflow）語法寫成硬體電路。

成果：

python 主要步驟：

1.將最小項（Minterms）與任思項（Don't cares）轉為二進位並依 1 的個數分組。

2.進行逐輪合併，找出所有質項（Prime Implicants, PIs）。

3.建立質項覆蓋表，選出必要質項（Essential Prime Implicants, EPIs）並覆蓋所有最小項，輸出最簡布林運算式。

Python 程式碼實作：

```
class QuineMcCluskey:
    def __init__(self, num_vars, minterms, dont_cares=None):
        self.num_vars = num_vars
```

```
        self.minterms = sorted(list(set(minterms)))
        self.dont_cares = sorted(list(set(dont_cares))) if dont_cares else []
        self.all_terms = sorted(self.minterms + self.dont_cares)

    def _to_binary(self, val):
        return bin(val)[2:].zfill(self.num_vars)

    def _count_ones(self, s):
        return s.count('1')

    def get_prime_implicants(self):
        # 步驟一：將所有項轉為二進位並依 1 的數量分組
        groups = {}
        for term in self.all_terms:
            b_str = self._to_binary(term)
            ones = self._count_ones(b_str)
            if ones not in groups:
```

```
            groups[ones] = {}
        groups[ones][b_str] = {term}

prime_implicants = set()

# 步驟二：疊代合併
while True:
    next_groups = {}
    marked = set()
    keys = sorted(groups.keys())

    for i in range(len(keys) - 1):
        g1 = groups[keys[i]]
        g2 = groups[keys[i+1]]
        for t1 in g1:
            for t2 in g2:
                # 檢查是否只有一個位元不同（Hamming 距離為 1）
```

```
diff = 0

diff_index = -1

for idx in range(self.num_vars):

    if t1[idx] != t2[idx]:

        diff += 1

        diff_index = idx



if diff == 1:

    #  合併並將不同位元置為  '-'

    new_term = t1[:diff_index] + '-' + t1[diff_index+1:]

    combined_originals = g1[t1].union(g2[t2])



    ones = self._count_ones(new_term)

    if ones not in next_groups:

        next_groups[ones] = {}

    next_groups[ones][new_term] = combined_originals
```

```
                                marked.add(t1)

                                marked.add(t2)



            # 收集未被標記的項目作為質項 (PIs)

            unmarked = set()

            for g in groups.values():

                for t in g:

                    if t not in marked:

                        prime_implicants.add((t, frozenset(g[t])))



            if not next_groups:

                break

            groups = next_groups



        return prime_implicants



def simplify(self):
```

```
pis = self.get_prime_implicants()



# 過濾掉只包含 don't cares 的質項

valid_pis = []

for pi, covered in pis:

    real_covered = covered.intersection(set(self.minterms))

    if real_covered:

        valid_pis.append((pi, real_covered))



# 步驟三：尋找必要質項 (EPIs) 與最佳覆蓋

pi_chart = {m: [] for m in self.minterms}

for idx, (pi, covered) in enumerate(valid_pis):

    for m in covered:

        pi_chart[m].append(idx)



selected_pis = set()

covered_minterms = set()
```

```
# 尋找 EPIs (若某個 minterm 只有一個 PI 可以覆蓋它)
for m, pi_indices in pi_chart.items():
    if len(pi_indices) == 1:
        selected_pis.add(pi_indices[0])

for idx in selected_pis:
    covered_minterms.update(valid_pis[idx][1])

# 若還有未被覆蓋的 minterm，使用貪婪法選出能覆蓋最多剩餘項的 PI
while covered_minterms != set(self.minterms):
    uncovered = set(self.minterms) - covered_minterms
    best_idx = -1
    max_cover = 0
    for idx, (pi, covered) in enumerate(valid_pis):
        if idx in selected_pis:
            continue
```

```
            intersection = covered.intersection(uncovered)
            if len(intersection) > max_cover:
                max_cover = len(intersection)
                best_idx = idx
        if best_idx != -1:
            selected_pis.add(best_idx)
            covered_minterms.update(valid_pis[best_idx][1])
        else:
            break

# 步驟四：將二進位格式轉回文字布林表達式（例如 A, B, C, D...）
result_expressions = []
var_names = [chr(ord('A') + i) for i in range(self.num_vars)]

for idx in selected_pis:
    pi_str = valid_pis[idx][0]
    term_str = []
```

```
                for i, char in enumerate(pi_str):
                    if char == '1':
                        term_str.append(var_names[i])
                    elif char == '0':
                        term_str.append(var_names[i] + "'")
                result_expressions.append("".join(term_str))

        return " + ".join(result_expressions)

# ==================== 測試範例 ====================
if __name__ == "__main__":
    # 範例：4 變數函數，給定 Minterms 與 Don't cares
    # f(A,B,C,D) = m(0, 2, 5, 6, 7, 8, 10, 12, 13) + d(15)
    NUM_VARS = 4
    MINTERMS = [0, 2, 5, 6, 7, 8, 10, 12, 13]
    DONT_CARES = [15]
```

```
    qm = QuineMcCluskey(NUM_VARS, MINTERMS, DONT_CARES)

    simplified_expression = qm.simplify()



    print("輸入最小項 (Minterms):", MINTERMS)

    print("任思項 (Don't cares):", DONT_CARES)

    print("----------------------------------------")

    print("化簡後的最佳布林運算式：", simplified_expression)
```

VHDL 主要步驟：

假設我們有一組 4 變數的最小項，透過 Q-M 化簡法得到了最簡布林運算式（例如：$F = A'B + CD + B'C'$），我們可以用以下的 VHDL 程式碼來實作這個組合邏輯電路：

VHDL 程式碼實作：

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;



-- 定義實體 (Entity)，宣告輸入與輸出接腳

entity QM_Simplified_Circuit is

    Port (

        A : in  STD_LOGIC;
```

```
        B : in    STD_LOGIC;
        C : in    STD_LOGIC;
        D : in    STD_LOGIC;
        F : out STD_LOGIC
    );
end QM_Simplified_Circuit;

-- 定義架構 (Architecture)，描述內部邏輯運算
architecture Dataflow of QM_Simplified_Circuit is
begin
    -- 假設 Q-M 化簡後的結果為：F = A'B + CD + B'C'
    -- 在 VHDL 中：
    -- NOT 運算為 「not」
    -- AND 運算為 「and」
    -- OR 運算為 「or」

    F <= (not A and B) or (C and D) or (not B and not C);
```

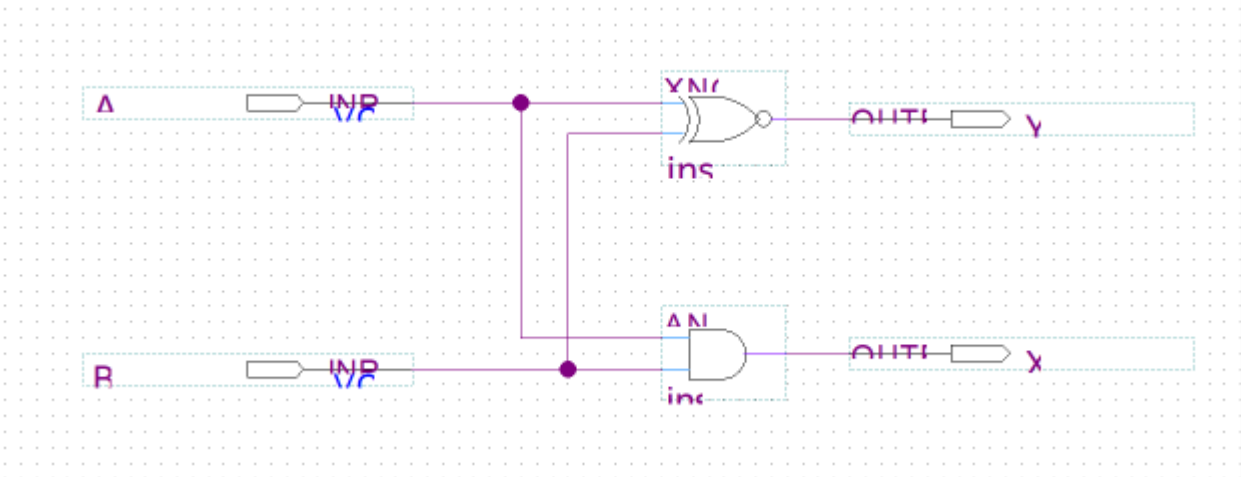
end Dataflow;

AI 工具協作過程：

1. 先將完整的題目「1. 請各組了解 Q-M 化簡法，並且運用 AI 工具設計 Q-M 化簡法後，了解相關程式概念，繳交程式說明與成果。」交給 gemini 分析。
2. 請 gemini「提供一段完整的 Python Q-M 實作程式碼範例」。
3. 請 gemini「用 vhdl 完成後續的部分」。

2. 請設計一個半加法器電路，但當加數相同被加數時，則 y 輸出為 1，進位值邏輯不變。(電路圖、真值表、卡諾圖)。

半加器電路圖：



真值表：

輸入 A	輸入 B	輸出 Y	進位 X
0	0	1	0
0	1	0	0
1	0	0	0
1	1	1	1

卡諾圖：

輸出 Y

A\B	0	1
0	1	0
1	0	1

進位 X

A\B	0	1
0	0	0
1	0	1